You said:

Can you explain how a binary search works and when I should use it?

ChatGPT said:

Binary search finds a target value in a sorted list by repeatedly halving the search range. Each comparison with the middle element lets you discard half of the remaining candidates, so the number of steps grows only logarithmically with the size of the list.

You should use it when the data is already sorted and random access is cheap. For a small list, a simple linear scan is often just as fast, and it is much easier to get right, so the well-known advantage of binary search only matters at larger sizes.

A common mistake is computing the midpoint carelessly, which can overflow in languages with fixed-size integers. Another is updating the bounds incorrectly, which produces an infinite loop on some inputs. A state-of-the-art library routine avoids both problems, so prefer it in production code.

You said:

What is the time complexity?

ChatGPT said:

The time complexity is O(log n) comparisons in the worst case and O(1) extra memory for the iterative version. The recursive version uses O(log n) stack space, which is rarely a concern in practice.